American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)
Department of Computer Science

Natural Language Processing
Mid-Term Project Report
Summer 2025-2026

Section: B
Group: 07

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name: Tanvir Mahatab Anan ID: 23-50807-1	Dataset1 Preparation, Vectorization TF-IDF, Tokenization, Apply SVM algorithm, Python environment creation
2.	Name: Jannatun Nesa Jerin ID: 23-50798-1	Documentation, stop words removal, Vectorization BoW Result Interpretation, Apply Naïve Bias algorithm, Dataset Description
3.	Name: MD Tanvir Islam Sarker Tamim ID: 23-50786-1	Dataset2 preparation, Text Preprocessing, Apply Logistic regression Algorithm, Stemming, Evaluation table creation
4.	Name: MD Mahmodul Hasan ID: 23-50794-1	Case folding, punctuation removal, Synonym substitution, Sample dataset creation

Dataset Description

Dataset 1: AG News Topic Classification Dataset Report

The AG News Topic Classification Dataset is a popular benchmark dataset for text & document classification. A subset of 2000 news articles was chosen and classified into four classes for this project:

Class 0: World
Class 1: Sports
Class 2: Business
Class 3: Sci/Tech

The textual news content is linked to one of the four categories for each sample. The key articles were pre-processed and encoded as a numerical representation of features based on Bag of Words (BoW) and TF-IDF to be classified using various machine learning algorithms.
Source:https://huggingface.co/datasets/fancyzhx/ag_news

Dataset 2: MSME Legal Dispute Classification Dataset

The MSME Legal Dispute Classification Dataset consists of legal dispute documents that are classified into six classes. In this project, 2,000 samples from four classes (0-3) were randomly chosen for document classification:

1. Class 0: Delayed Payment (No Dispute)
2. Class 1: Quality Dispute
3. Class 2: No Formal Contract
4. Class 3: Partial Payment Dispute

Legal details include statements of claim, buyer responses, case summaries, contract/agreement information and payment information. These documents were preprocessed and represented in the form of Bag of Words (BoW) and TF-IDF models and classified by applying various machine learning algorithms.
Source:https://huggingface.co/datasets/abhinavdread/msme-legal-dispute-classification-dataset

Project Implementation Detail

Libraries:

Data Handling

- import pandas as pd
- import numpy as np

Test Processing

- import string
- import nltk from nltk.tokenizeimport word_tokenize
- from nltk.stem import PorterStemmer
- from nltk.corpus import stopwords

Feature Extraction

- from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

Data Splitting

- from sklearn.model_selection import train_test_split

Machine Learning Models

- from sklearn.naive_bayes import MultinomialNB
- from sklearn.linear_model import LogisticRegression
- from sklearn.svm import LinearSVC

Model Evaluation

- from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_scoreconfusion_matrix, ConfusionMatrixDisplay

Data Visualization

- import matplotlib.pyplot as plt

Load Datasets:

```
df1 = pd.read_excel('/content/AG_NEWS.xlsx')
df2 = pd.read_excel('/content/MSME_DATASET.xlsx')

df1.dropna(how='any', inplace=True)
df2.dropna(how='any', inplace=True)
text_col = 'text'
label_col = 'label'

X1 = df1[text_col].astype(str)
y1 = df1[label_col]

X2 = df2[text_col].astype(str)
y2 = df2[label_col]

display(df1.head(10))
display(df2.head(10))
```

Description:

The two datasets, AG_NEWS and MSME_DATASET are loaded into pandas, missing values are dropped and the text and label columns are split out for processing. At last, the first 10 rows of each of the datasets are shown.

Tokenization

First step in an NLP pipeline is tokenization. It breaks a text down in smaller meaningful units called tokens so that a machine can more easily understand and analyze that text. Tokens can be sentences, words, characters, or subwords.

Word tokenization is performed on X1 and X2 in this task. Word tokenization is the process of dividing the text into words or tokens. X1 and X2 are used to take each text entry into the function word_tokenize() and store the tokenized text in the variables tokens_1 and tokens_2, respectively.

Code of Tokenization:

```
def apply_tokenization(text):
    return word_tokenize(text)

tokens_1 = X1.apply(apply_tokenization)
tokens_2 = X2.apply(apply_tokenization)

print("Word found:Dataset 1 ", len(tokens_1))
print(tokens_1)

print("Word found:Dataset 2 ", len(tokens_2))
```

```
print(tokens_2)
```

Sample output of Tokenization:

```
Word found:Dataset 1  2000
0       [Wall, St., Bears, Claw, Back, Into, the, Blac...
1       [Carlyle, Looks, Toward, Commercial, Aerospace...
2       [Oil, and, Economy, Cloud, Stocks, ', Outlook,...
3       [Iraq, Halts, Oil, Exports, from, Main, Southe...
4       [Oil, prices, soar, to, all-time, record, ,, p...
                              ...
1995    [Google, Interview, Is, Draw, for, Latest, Pla...
1996    [Williamson, Gets, Third, Opinion, on, Elbow, ...
1997    [Iran, Threatens, Israel, on, Nuclear, Reactor...
1998    [Grim, funeral, service, held, for, Burundi, m...
1999    [Borders, Profits, Rise, ,, Raises, Outlook, N...
Name: text, Length: 2000, dtype: object
Word found:Dataset 2  2000
0       [ASSISTANT, DIRECTOR, ,, ENFORCEMENT, DIRECTOR...
1       [State, of, Maharashtra, ), Office, Notes, ,, ...
2       [Bangalore, District, Court, M/S, Cauvery, Dis...
3       [M/s, Suri, Electricals, and, Ceramics, .....,...
4       [Delhi, High, Court, Metal, Forgings, Pvt, ., ...
                              ...
1995    [Allahabad, High, Court, Anil, Kumar, Mishra, ...
1996    [Bombay, High, Court, Dodale, Electro, Instrum...
1997    [State, Represented, by, Deputy, Superintenden...
1998    [Telangana, High, Court, Security, Printing, P...
1999    [M/s, ., Dhanashree, Electronics, Limited, Mr....
Name: text, Length: 2000, dtype: object
```

Code description:

- apply_tokenization(text) is used to tokenize the text.
- word_tokenize(text) returns a list of words/tokens in the text..
- The function is used for X1 and X2.
- X1 is stored in tokens_1 as a token.The tokens of X2 are stored in tokens_2.
- X2 is stored in the tokenized form as tokens_2..
- Lastly, print() prints out the results of both datasets after tokenization.

Case folding

Case folding is one of word normalization methods used to normalize words/tokens to lower case. In this task, the lowercase version of each of the tokens in tokens_1 and tokens_2 is determined by calling the lower() function.

Code for Case folding:

```
def apply_case_folding(tokens):
    return [x.lower() for x in tokens]

tokens_1 = tokens_1.apply(apply_case_folding)
tokens_2 = tokens_2.apply(apply_case_folding)

print("Lower case:Dataset 1 ", len(tokens_1))
print(tokens_1)

print("Lower case:Dataset 2 ", len(tokens_2))
print(tokens_2)
```

Sample Output of Case folding:

```
Lower case:Dataset 1  2000
0       [wall, st., bears, claw, back, into, the, blac...
1       [carlyle, looks, toward, commercial, aerospace...
2       [oil, and, economy, cloud, stocks, ', outlook,...
3       [iraq, halts, oil, exports, from, main, southe...
4       [oil, prices, soar, to, all-time, record, ,, p...
                            ...
1995    [google, interview, is, draw, for, latest, pla...
1996    [williamson, gets, third, opinion, on, elbow, ...
1997    [iran, threatens, israel, on, nuclear, reactor...
1998    [grim, funeral, service, held, for, burundi, m...
1999    [borders, profits, rise, ,, raises, outlook, n...
Name: text, Length: 2000, dtype: object
Lower case:Dataset 2  2000
0       [assistant, director, ,, enforcement, director...
1       [state, of, maharashtra, ), office, notes, ,, ...
2       [bangalore, district, court, m/s, cauvery, dis...
3       [m/s, suri, electricals, and, ceramics, .....,...
4       [delhi, high, court, metal, forgings, pvt, ., ...
                            ...
1995    [allahabad, high, court, anil, kumar, mishra, ...
1996    [bombay, high, court, dodale, electro, instrum...
1997    [state, represented, by, deputy, superintenden...
1998    [telangana, high, court, security, printing, p...
1999    [m/s, ., dhanashree, electronics, limited, mr....
Name: text, Length: 2000, dtype: object
```

Code description:

- apply_case_folding() is used to convert the tokens into lowercase.
- x.lower(): Transforms all the tokens in x to lower case.
- The function is called on the two tokens: tokens_1 and tokens_2.
- The lowercase results are stored once again in the tokens_1 and tokens_2.
- Finally, print() prints the lower case forms of both datasets.

Synonym Substitution

Using a synonym to replace a word. In this task, certain words in tokens_1 and tokens_2 are replaced by words as part of synonyms dictionary. This will normalize the text and standardize similar forms of words.

Code for Synonym Substitution:

```
synonyms = {
    "won't": "will not",
    "can't": "cannot",
    "n't": "not",
    "good": "nice",
    "bad": "poor",
    "big": "large",
    "small": "little",
    "happy": "glad",
    "quick": "fast",
    "begin": "start",
    "prices": "price"
}

def apply_synonym_substitution(tokens):
    replaced_tokens = []
```

```
    for t in tokens:
        if t in synonyms:
            t = synonyms[t]

        replaced_tokens.append(t)
return replaced_tokens

tokens_1 = tokens_1.apply(apply_synonym_substitution)
tokens_2 = tokens_2.apply(apply_synonym_substitution)

print("Substitute Synonyms: Dataset 1 ", len(tokens_1))
print(tokens_1)

print("Substitute Synonyms: Dataset 2 ", len(tokens_2))
print(tokens_2)
```

Sample output of Synonym Substitution:

```
Substitute Synonyms: Dataset 1  2000
0       [wall, st., bears, claw, back, into, the, blac...
1       [carlyle, looks, toward, commercial, aerospace...
2       [oil, and, economy, cloud, stocks, ', outlook,...
3       [iraq, halts, oil, exports, from, main, southe...
4       [oil, price, soar, to, all-time, record, ,, po...
                              ...
1995    [google, interview, is, draw, for, latest, pla...
1996    [williamson, gets, third, opinion, on, elbow, ...
1997    [iran, threatens, israel, on, nuclear, reactor...
1998    [grim, funeral, service, held, for, burundi, m...
1999    [borders, profits, rise, ,, raises, outlook, n...
Name: text, Length: 2000, dtype: object
Substitute Synonyms: Dataset 2  2000
0       [assistant, director, ,, enforcement, director...
1       [state, of, maharashtra, ), office, notes, ,, ...
2       [bangalore, district, court, m/s, cauvery, dis...
3       [m/s, suri, electricals, and, ceramics, .....,...
4       [delhi, high, court, metal, forgings, pvt, ., ...
                              ...
1995    [allahabad, high, court, anil, kumar, mishra, ...
1996    [bombay, high, court, dodale, electro, instrum...
1997    [state, represented, by, deputy, superintenden...
1998    [telangana, high, court, security, printing, p...
1999    [m/s, ., dhanashree, electronics, limited, mr....
Name: text, Length: 2000, dtype: object
```

Code description:

- The synonyms dictionary contains the dictionary of original word and replacement word.
- The selected words are replaced by using the function apply_synonym_substitution().
- Each token is checked in turn using the for loop.
- If a token is in the synonyms dictionary it will be replaced with the word given.
- The tokens that are replaced are placed in replaced_tokens.
- The function is applied to tokens_1 and tokens_2.
- Finally, print() prints out the results for both datasets.

Stemming

Stemming refers to the removal of prefixes and/or suffixes from words and transforming them into their root or basic form known as the stem..
PorterStemmer is used to stem the words in tokens_1 and tokens_2 for this task.

Code for Stemming:

```
 stemmer = PorterStemmer()

def apply_stemming_and_join(tokens):
    stemmed_words = [stemmer.stem(word) for word in tokens]
    return " ".join(stemmed_words)

X1_clean = tokens_1.apply(apply_stemming_and_join)
X2_clean = tokens_2.apply(apply_stemming_and_join)

print("Stems: Dataset 1 ", len(tokens_1))
print(tokens_1)

print("Stems: Dataset 2 ", len(tokens_2))
print(tokens_2)
```

Sample output of Stemming:

```
Stems: Dataset 1  2000
0       [wall, st., bears, claw, back, into, the, blac...
1       [carlyle, looks, toward, commercial, aerospace...
2       [oil, and, economy, cloud, stocks, ', outlook,...
3       [iraq, halts, oil, exports, from, main, southe...
4       [oil, price, soar, to, all-time, record, ,, po...
                              ...
1995    [google, interview, is, draw, for, latest, pla...
1996    [williamson, gets, third, opinion, on, elbow, ...
1997    [iran, threatens, israel, on, nuclear, reactor...
1998    [grim, funeral, service, held, for, burundi, m...
1999    [borders, profits, rise, ,, raises, outlook, n...
Name: text, Length: 2000, dtype: object
Stems: Dataset 2  2000
0       [assistant, director, ,, enforcement, director...
1       [state, of, maharashtra, ), office, notes, ,, ...
2       [bangalore, district, court, m/s, cauvery, dis...
3       [m/s, suri, electricals, and, ceramics, .....,...
4       [delhi, high, court, metal, forgings, pvt, ., ...
                              ...
1995    [allahabad, high, court, anil, kumar, mishra, ...
1996    [bombay, high, court, dodale, electro, instrum...
1997    [state, represented, by, deputy, superintenden...
1998    [telangana, high, court, security, printing, p...
1999    [m/s, ., dhanashree, electronics, limited, mr....
Name: text, Length: 2000, dtype: object
```

Code description :

- The stemming has been done using PorterStemmer()..
- apply_stemming_and_join() is used to stem the tokens.
- The stemmer stemmer.stem() returns the stem of each word.
- The dictionary of the stemmed words are stored in the dictionary stemmed_words.
- The function “”.join() glues the stemmed words together and creates one text.
- The results are saved in X1_clean and X2_clean.
- Finally, print() prints the result of both datasets.

Punctuation removal

Punctuation removal is the process of removing punctuation marks and special characters from text. This task involves removing punctuation from the tokens_1 and tokens_2, making the data cleaner and easier to be processed.

Code for Punctuation removal:

```
def apply_punctuation_removal(tokens):
    text_remove_mapping = str.maketrans("", "", string.punctuation)
    clean_tokens = [t.translate(text_remove_mapping) for t in tokens]
    return [t for t in clean_tokens if t.strip() != ""]

tokens_1 = tokens_1.apply(apply_punctuation_removal)
tokens_2 = tokens_2.apply(apply_punctuation_removal)

print("Puncuation Remove : Dataset 1 ", len(tokens_1))
print(tokens_1)

print("Puncuation Remove : Dataset 2 ", len(tokens_2))
print(tokens_2)
```

Sample output of Punctuation removal:

```
Puncuation Remove : Dataset 1  2000
0       [wall, st, bears, claw, back, into, the, black...
1       [carlyle, looks, toward, commercial, aerospace...
2       [oil, and, economy, cloud, stocks, outlook, re...
3       [iraq, halts, oil, exports, from, main, southe...
4       [oil, price, soar, to, alltime, record, posing...
                              ...
1995    [google, interview, is, draw, for, latest, pla...
1996    [williamson, gets, third, opinion, on, elbow, ...
1997    [iran, threatens, israel, on, nuclear, reactor...
1998    [grim, funeral, service, held, for, burundi, m...
1999    [borders, profits, rise, raises, outlook, new,...
Name: text, Length: 2000, dtype: object
Puncuation Remove : Dataset 2  2000
0       [assistant, director, enforcement, directorate...
1       [state, of, maharashtra, office, notes, office...
2       [bangalore, district, court, ms, cauvery, dist...
3       [ms, suri, electricals, and, ceramics, respond...
4       [delhi, high, court, metal, forgings, pvt, ltd...
                              ...
1995    [allahabad, high, court, anil, kumar, mishra, ...
1996    [bombay, high, court, dodale, electro, instrum...
1997    [state, represented, by, deputy, superintenden...
1998    [telangana, high, court, security, printing, p...
1999    [ms, dhanashree, electronics, limited, mr, ans...
Name: text, Length: 2000, dtype: object
```

Code description:

- The punctuation is removed by using a function called apply_punctuation_removal().
- The removed punctuation marks are specified in a rule that is created with str.maketrans().
- translate() will strip out the punctuation from each token.
- clean_tokens is a list of the cleaned tokens.
- Empty tokens are removed from the result.
- The function is called on the tokens_1 and tokens_2 lists.

- Finally, print() shows the results of both datasets..

Stop words removal
Stop words are common words that have very little meaning on their own, such as “a”, “an”, “the”, and “in”. In this task, English stop words are removed from tokens_1 and tokens_2 to keep the more useful words in the text.

Code for Stop words removal:

```
stop_words = set(stopwords.words("english"))

def apply_stopwords_removal(tokens):
    return [word for word in tokens if word not in stop_words]

X1_clean = tokens_1.apply(apply_stopwords_removal)
X2_clean = tokens_2.apply(apply_stopwords_removal)

print("Stop words remove: Dataset 1 ", len(X1_clean))
print(tokens_1)

print("Stop words remove: Dataset 2 ", len(X2_clean))
print(tokens_2)
```

Sample output of Stop words removal:

```
Stop words remove: Dataset 1  2000
0       [wall, st, bears, claw, back, into, the, black...
1       [carlyle, looks, toward, commercial, aerospace...
2       [oil, and, economy, cloud, stocks, outlook, re...
3       [iraq, halts, oil, exports, from, main, southe...
4       [oil, price, soar, to, alltime, record, posing...
                              ...
1995    [google, interview, is, draw, for, latest, pla...
1996    [williamson, gets, third, opinion, on, elbow, ...
1997    [iran, threatens, israel, on, nuclear, reactor...
1998    [grim, funeral, service, held, for, burundi, m...
1999    [borders, profits, rise, raises, outlook, new,...
Name: text, Length: 2000, dtype: object
Stop words remove: Dataset 2  2000
0       [assistant, director, enforcement, directorate...
1       [state, of, maharashtra, office, notes, office...
2       [bangalore, district, court, ms, cauvery, dist...
3       [ms, suri, electricals, and, ceramics, respond...
4       [delhi, high, court, metal, forgings, pvt, ltd...
                              ...
1995    [allahabad, high, court, anil, kumar, mishra, ...
1996    [bombay, high, court, dodale, electro, instrum...
1997    [state, represented, by, deputy, superintenden...
1998    [telangana, high, court, security, printing, p...
1999    [ms, dhanashree, electronics, limited, mr, ans...
Name: text, Length: 2000, dtype: object
```

Code description

- stop_words stores the common English stop words.
- apply_stopwords_removal() is used to remove stop words.

- Each word is checked with the stop word list.
- Stop words are removed from the tokens.
- The cleaned results are stored in X1_clean and X2_clean.
- Finally, print() shows the results of both datasets.

Vectorization (BoW)
BoW is a text representation technique that converts text into numerical vectors based on word frequency. It creates a vocabulary of unique words and represents each document by counting how often each word appears, while ignoring word order and grammar.

Code BoW:

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
count_vector = CountVectorizer()
X1_train_bow = count_vector.fit_transform(X1_train)
X1_test_bow = count_vector.transform(X1_test)

count_vector_2 = CountVectorizer()
X2_train_bow = count_vector_2.fit_transform(X2_train)
X2_test_bow = count_vector_2.transform(X2_test)
```

Code description
This code applies Bag of Words (BoW) using CountVectorizer to convert the text from Dataset 1 and Dataset 2 into numerical feature vectors. The vectorizer learns the vocabulary from the training data using fit_transform() and converts the test data using transform() with the same vocabulary.

Vectorization (TF-IDF)
TF-IDF is a text representation method that transforms the text to numerical vectors with respect to word importance. It is able to assign more weight to the words that are used in a specific document but less frequently in the whole set of documents, thereby emphasizing more important words for classification..

Code TF-IDF:

```
tfidf_vectorizer = TfidfVectorizer()
X1_train_tfidf = tfidf_vectorizer.fit_transform(X1_train)
X1_test_tfidf = tfidf_vectorizer.transform(X1_test)

tfidf_vectorizer_2 = TfidfVectorizer()
X2_train_tfidf = tfidf_vectorizer_2.fit_transform(X2_train)
X2_test_tfidf = tfidf_vectorizer_2.transform(X2_test)
```

Code description
This code uses the TF-IDF (Term Frequency–Inverse Document Frequency) to transform text data from two datasets (Dataset 1 and Dataset 2) into numerical feature vectors according to the importance of the words. It fits the model using fit_transform() with the training data and transforms the test data using the same set of features it has learned from fit_transform() with transform()

Train_and_Test_Data

The portion of the data that is used to train the machine learning model. Based on this data, the model learns patterns and relationships that are used for prediction. The unseen data that is used for testing the performance of the trained model and for determining the accuracy of the model in classifying new data is called testing data.

Code for Dataset Splittng:

```
from sklearn.model_selection import train_test_split

X1_train, X1_test, y1_train, y1_test = train_test_split(X1_clean, y1, test_size=0.2,
random_state=42)
X2_train, X2_test, y2_train, y2_test = train_test_split(X2_clean, y2, test_size=0.2,
random_state=42)
```

Code Description:

This code divides the Dataset 1 and Dataset 2 into training and test sets using train_test_split(). It operates with 80% of the data for training and 20% for testing. You can obtain the same split each time by using the random_state=42 option.

Model Training and Evaluation

This is the process of using preprocessed and vectorized text data (BoW or TF-IDF) to train machine learning algorithms. The model is trained with the provided documents and it takes the learned patterns to classify new documents into their classes and it shows its training performance via performance metrices.

Code of Model Training and Evaluation:

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, ConfusionMatrixDisplay

def evaluate_and_print(model, X_train, y_train, X_test, y_test, model_name, rep, dataset):
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    acc = accuracy_score(y_test, predictions)
    prec = precision_score(y_test, predictions, average='weighted', zero_division=0)
    rec = recall_score(y_test, predictions, average='weighted', zero_division=0)
    f1 = f1_score(y_test, predictions, average='weighted', zero_division=0)
    cm = confusion_matrix(y_test, predictions)

    print(model_name, "|", rep, "|", dataset)
```

```
print("Accuracy: ", acc)
print("Precision:", prec)
print("Recall:   ", rec)
print("F1 Score: ", f1)
print("-" * 40)
```

Code Description
This code is used to train and test classification models. It is used to predict the test data (model.predict) and computes four performance metrics namely Accuracy, Precision, Recall and F1 Score. It also creates a Confusion Matrix to analyze the accuracy of classification of the model..

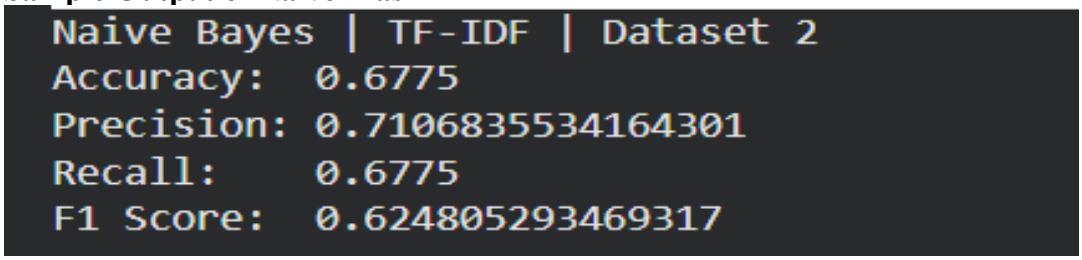
Application of Algorithms

Naïve Bias
Naive Bayes is a simple algorithm of machine learning, widely used for text classification. It determines the probability of each class using the occurrence of words within a document and classifies the document to the class with the highest probability. Quick, easy and effective for BoW and TF-IDF features.
.

Code
```
nb_model = MultinomialNB()

nb_d1_bow = evaluate_and_print(nb_model, X1_train_bow, y1_train, X1_test_bow, y1_test, "Naive Bayes", "BoW", "Dataset 1")
nb_d1_tfidf= evaluate_and_print(nb_model, X1_train_tfidf, y1_train, X1_test_tfidf, y1_test, "Naive Bayes", "TF-IDF", "Dataset 1")
nb_d2_bow = evaluate_and_print(nb_model, X2_train_bow, y2_train, X2_test_bow, y2_test, "Naive Bayes", "BoW", "Dataset 2")
nb_d2_tfidf= evaluate_and_print(nb_model, X2_train_tfidf, y2_train, X2_test_tfidf, y2_test, "Naive Bayes", "TF-IDF", "Dataset 2")
```

Sample Output of Naïve Bias



Code Description
This code trains and evaluates the Multinomial Naive Bayes model using both BoW and TF-IDF features for both Dataset 1 and Dataset 2. It evaluates the performance of the proposed model on the basis of Accuracy, Precision, Recall and F1 Score for each of the datasets and feature representations.
.

Logistic Regression
A supervised classification algorithm feature for learns weights and decision boundaries to classify documents into different categories; classification is effective for text with BoW and TF-IDF features.

Code
```
from sklearn.linear_model import LogisticRegression

lr_model = LogisticRegression(max_iter=1000)

lr_d1_bow = evaluate_and_print(lr_model, X1_train_bow, y1_train, X1_test_bow, y1_test, "Logistic Regression", "BoW", "Dataset 1")
lr_d1_tfidf = evaluate_and_print(lr_model, X1_train_tfidf, y1_train, X1_test_tfidf, y1_test, "Logistic Regression", "TF-IDF", "Dataset 1")
lr_d2_bow = evaluate_and_print(lr_model, X2_train_bow, y2_train, X2_test_bow, y2_test, "Logistic Regression", "BoW", "Dataset 2")
lr_d2_tfidf = evaluate_and_print(lr_model, X2_train_tfidf, y2_train, X2_test_tfidf, y2_test, "Logistic Regression", "TF-IDF", "Dataset 2")
```

Sample Output of Logistic Regression
```
Logistic Regression | BoW | Dataset 1
Accuracy:  0.79
Precision: 0.7924187806873977
Recall:    0.79
F1 Score:  0.7879634127610728
----------------------------------------
```

Code Description
This code trains and test Logistic Regression Classifier on both Dataset 1 and Dataset 2 with both the feature BoW and TF-IDF. It evaluates the performance of the model based on Accuracy, datasets and feature representations are across different Precision, Recall and F1 Score.

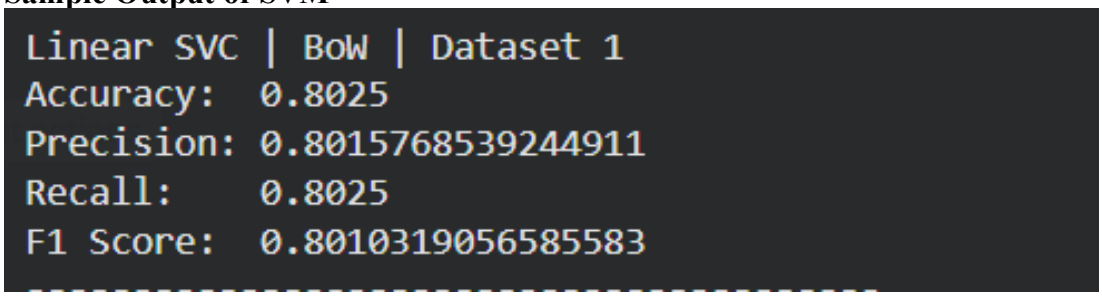
Support Vector Machine (SVM)
SVM is a supervised machine learning algorithm which aims to derive the optimal class separation decision boundary (hyperplane) between the classes. It's very useful with high dimensional text data and it works quite well with BoW and TF-IDF features.
Code
```
from sklearn.svm import LinearSVC

svm_model = LinearSVC()

svm_d1_bow = evaluate_and_print(svm_model, X1_train_bow, y1_train, X1_test_bow, y1_test, "Linear SVC", "BoW", "Dataset 1")
svm_d1_tfidf = evaluate_and_print(svm_model, X1_train_tfidf, y1_train, X1_test_tfidf, y1_test, "Linear SVC", "TF-IDF", "Dataset 1")
```

```
svm_d2_bow = evaluate_and_print(svm_model, X2_train_bow, y2_train, X2_test_bow,
y2_test, "Linear SVC", "BoW", "Dataset 2")
svm_d2_tfidf = evaluate_and_print(svm_model, X2_train_tfidf, y2_train, X2_test_tfidf,
y2_test, "Linear SVC", "TF-IDF", "Dataset 2")
```

Sample Output of SVM



Code Description

This code trains and evaluates SVM model on Dataset 1 and Dataset 2 with BoW features and TF-IDF features. It checks the performance of the model with accuracy, precision, recall, and f1 score for each dataset and feature representation.

Algorithm	Representation	Dataset 1	Dataset 2
Naive Bayes	**BoW**	Accuracy: 0.825 Precession: 0.827 Recall: 0.825 F1 Score: 0.823	Accuracy: 0.8325 Precession: 0.8301 Recall: 0.8325 F1 Score: 0.8280
	TF-IDF	Accuracy: 0.763 Precession: 0.8001 Recall: 0.763 F1 Score: .754	Accuracy: 0.6775 Precession: 0.7106 Recall: 0.6775 F1 Score: 0.6248
Logistic Regression	**BoW**	Accuracy: 0.79 Precession: 0.7924 Recall: 0.79 F1 Score: 0.788	Accuracy: 0.8525 Precession: 0.8511 Recall: 0.8525 F1 Score: 0.8506
	TF-IDF	Accuracy: 0.775 Precession: 0.8015 Recall: 0.775 F1 Score: 0.7696	Accuracy: 0.815 Precession: 0.8128 Recall: 0.815 F1 Score: 0.8075
Support Vector Machine	**BoW**	Accuracy: 0.8025 Precession: 0.8015 Recall: 0.8025 F1 Score: 0.8010	Accuracy: 0.8475 Precession: 0.8075 Recall: 0.8475 F1 Score: 0.8453
	TF-IDF	Accuracy: 0.83 Precession: 0.8323 Recall: 0.83 F1 Score: **0.8274**	Accuracy: 0.8675 Precession: 0.867 Recall: 0.8675 F1 Score: **0.8648**

DESCRIPTION

This table displays the performance of Naive Bayes, Logistic Regression, and Support Vector Machine (SVM) two different text representation methods, namely Bag of Words (BoW) and TF-IDF in both two datasets. To enable machine learning algorithms to work on text, it is converted into numerical features using BoW and TF-IDF. BoW is text based on word frequency, TF-IDF is informative words more important. Accuracy, Precision, Recall and F1 Score are used for evaluating the models.

Naive Bayes: BoW gives better results than TF-IDF for both datasets. There is no difference between the accuracy of BoW on Dataset 1 and on Dataset 2.The accuracy of BoW is 0.825 on Dataset 1 and 0.8325 on Dataset 2. The accuracy for TF-IDF is 0.763 in Dataset 1 and 0.6775 in Dataset 2. Thus, BoW is more effective for Naive Bayes in this experiment.

Logistic Regression: BoW is also outperforming TF-IDF. The accuracy for Dataset 1 and Dataset 2 with BoW is 0.7900 and 0.8525, respectively. The accuracy is 0.7750, and 0.8150, respectively, with TF-IDF. Hence, BoW is the best in Logistic Regression for Dataset 2.

Unlike the other two algorithms, Support Vector Machine (SVM) has better performance in TF-IDF. In the case of BoW, the accuracy of Dataset 1 and Dataset 2 are 0.8025 and 0.8475 respectively. The accuracies with TF-IDF are 0.8300 and 0.8675, respectively. When using Logistic Regression and SVM, we can see that Dataset 2 performs better in comparison to Dataset 1. But Naive Bayes (TF-IDF) works better for Dataset 1 than for Dataset 2.

Overall Best Result: SVM with TF-IDF on Dataset 2 gives the best result with the following parameters: Accuracy = 0.8675, Precision = 0.8670, Recall = 0.8675 and F1 Score = 0.8648.

Thus, it can be concluded that the selection of text representation technique influences the performance of the machine learning algorithms, as seen in the results. BoW is better with Naive Bayes and Logistic Regression, and TF-IDF is better with SVM in this experiment. The combination SVM/TF-IDF on Dataset 2 is the most effective combination overall.

Project Code

```
import pandas as pd
import numpy as np
import string
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')



df1 = pd.read_excel('/content/AG_NEWS.xlsx')
df2 = pd.read_excel('/content/MSME_DATASET.xlsx')

df1.dropna(how='any', inplace=True)
df2.dropna(how='any', inplace=True)
text_col = 'text'
label_col = 'label'

X1 = df1[text_col].astype(str)
y1 = df1[label_col]

X2 = df2[text_col].astype(str)
y2 = df2[label_col]

display(df1.head(10))
display(df2.head(10))




def apply_tokenization(text):
    return word_tokenize(text)
```

```
tokens_1 = X1.apply(apply_tokenization)
tokens_2 = X2.apply(apply_tokenization)
print("Word found:Dataset 1 ", len(tokens_1))
print(tokens_1)
print("Word found:Dataset 2 ", len(tokens_2))
print(tokens_2)



def apply_case_folding(tokens):
    return [x.lower() for x in tokens]

tokens_1 = tokens_1.apply(apply_case_folding)
tokens_2 = tokens_2.apply(apply_case_folding)
print("Lower case:Dataset 1 ", len(tokens_1))
print(tokens_1)
print("Lower case:Dataset 2 ", len(tokens_2))
print(tokens_2)



synonyms = {
    "won't": "will not",
    "can't": "cannot",
    "n't": "not",
    "good": "nice",
    "bad": "poor",
    "big": "large",
    "small": "little",
    "happy": "glad",
    "quick": "fast",
    "begin": "start",
    "prices" : "price"
}

def apply_synonym_substitution(tokens):
    replaced_tokens = []

    for t in tokens:
        if t in synonyms:
            t = synonyms[t]

        replaced_tokens.append(t)

    return replaced_tokens

tokens_1 = tokens_1.apply(apply_synonym_substitution)
tokens_2 = tokens_2.apply(apply_synonym_substitution)
print("Substitute Synonyms: Dataset 1 ", len(tokens_1))
```

```
print(tokens_1)
print("Substitute Synonyms: Dataset 2 ", len(tokens_2))
print(tokens_2)




stemmer = PorterStemmer()

def apply_stemming_and_join(tokens):
    stemmed_words = [stemmer.stem(word) for word in tokens]
    return " ".join(stemmed_words)

X1_clean = tokens_1.apply(apply_stemming_and_join)
X2_clean = tokens_2.apply(apply_stemming_and_join)
print("Stems: Dataset 1 ", len(tokens_1))
print(tokens_1)
print("Stems: Dataset 2 ", len(tokens_2))
print(tokens_2)



def apply_punctuation_removal(tokens):
    text_remove_mapping = str.maketrans("", "", string.punctuation)
    clean_tokens = [t.translate(text_remove_mapping) for t in tokens]
    return [t for t in clean_tokens if t.strip() != ""]

tokens_1 = tokens_1.apply(apply_punctuation_removal)
tokens_2 = tokens_2.apply(apply_punctuation_removal)
print("Puncuation Remove : Dataset 1 ", len(tokens_1))
print(tokens_1)
print("Puncuation Remove : Dataset 2 ", len(tokens_2))
print(tokens_2)




stop_words = set(stopwords.words("english"))

def apply_stopwords_removal(tokens):
    return [word for word in tokens if word not in stop_words]

X1_clean = tokens_1.apply(apply_stopwords_removal)
X2_clean = tokens_2.apply(apply_stopwords_removal)
print("Stop words remove: Dataset 1 ", len(X1_clean))
print(tokens_1)
```

```
print("Stop words remove: Dataset 2 ", len(X2_clean))
print(tokens_2)
```

```
from sklearn.model_selection import train_test_split

X1_train, X1_test, y1_train, y1_test = train_test_split(X1_clean, y1, test_size=0.2,
random_state=42)
X2_train, X2_test, y2_train, y2_test = train_test_split(X2_clean, y2, test_size=0.2,
random_state=42)
```

```
X1_train = X1_train.apply(lambda x: " ".join(x) if isinstance(x, list) else str(x))
X1_test = X1_test.apply(lambda x: " ".join(x) if isinstance(x, list) else str(x))
X2_train = X2_train.apply(lambda x: " ".join(x) if isinstance(x, list) else str(x))
X2_test = X2_test.apply(lambda x: " ".join(x) if isinstance(x, list) else str(x))
```

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
count_vector = CountVectorizer()
X1_train_bow = count_vector.fit_transform(X1_train)
X1_test_bow = count_vector.transform(X1_test)

count_vector_2 = CountVectorizer()
X2_train_bow = count_vector_2.fit_transform(X2_train)
X2_test_bow = count_vector_2.transform(X2_test)
```

```
tfidf_vectorizer = TfidfVectorizer()
X1_train_tfidf = tfidf_vectorizer.fit_transform(X1_train)
X1_test_tfidf = tfidf_vectorizer.transform(X1_test)

tfidf_vectorizer_2 = TfidfVectorizer()
X2_train_tfidf = tfidf_vectorizer_2.fit_transform(X2_train)
X2_test_tfidf = tfidf_vectorizer_2.transform(X2_test)
```

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, ConfusionMatrixDisplay

def evaluate_and_print(model, X_train, y_train, X_test, y_test, model_name, rep, dataset):
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    acc = accuracy_score(y_test, predictions)
    prec = precision_score(y_test, predictions, average='weighted', zero_division=0)
    rec = recall_score(y_test, predictions, average='weighted', zero_division=0)
    f1 = f1_score(y_test, predictions, average='weighted', zero_division=0)
    cm = confusion_matrix(y_test, predictions)

    print(model_name, "|", rep, "|", dataset)
    print("Accuracy: ", acc)
    print("Precision:", prec)
    print("Recall:   ", rec)
    print("F1 Score: ", f1)
    print("-" * 40)

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    cm_display = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=model.classes_)
    cm_display.plot(cmap=plt.cm.Blues, ax=ax1)
    ax1.set_title(f"Confusion Matrix: {model_name} ({rep})")

    metrics_names = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
    metrics_values = [acc, prec, rec, f1]

    bars = ax2.bar(metrics_names, metrics_values, color=['#4CAF50', '#2196F3', '#FFC107',
'#F44336'])
    ax2.set_ylim(0, 1.1)
    ax2.set_title(f"Performance Metrics: {model_name} ({rep})")
    ax2.set_ylabel("Score")

    for bar in bars:
        yval = bar.get_height()
        ax2.text(bar.get_x() + bar.get_width()/2, yval + 0.02, round(yval, 3), ha='center',
va='bottom', fontweight='bold')

    plt.tight_layout()
    plt.show()

    return round(acc, 4)
```

```
from sklearn.naive_bayes import MultinomialNB

nb_model = MultinomialNB()

nb_d1_bow = evaluate_and_print(nb_model, X1_train_bow, y1_train, X1_test_bow, y1_test,
"Naive Bayes", "BoW", "Dataset 1")
nb_d1_tfidf = evaluate_and_print(nb_model, X1_train_tfidf, y1_train, X1_test_tfidf, y1_test,
"Naive Bayes", "TF-IDF", "Dataset 1")
nb_d2_bow = evaluate_and_print(nb_model, X2_train_bow, y2_train, X2_test_bow, y2_test,
"Naive Bayes", "BoW", "Dataset 2")
nb_d2_tfidf = evaluate_and_print(nb_model, X2_train_tfidf, y2_train, X2_test_tfidf, y2_test,
"Naive Bayes", "TF-IDF", "Dataset 2")
```

```
from sklearn.linear_model import LogisticRegression

lr_model = LogisticRegression(max_iter=1000)

lr_d1_bow = evaluate_and_print(lr_model, X1_train_bow, y1_train, X1_test_bow, y1_test,
"Logistic Regression", "BoW", "Dataset 1")
lr_d1_tfidf = evaluate_and_print(lr_model, X1_train_tfidf, y1_train, X1_test_tfidf, y1_test,
"Logistic Regression", "TF-IDF", "Dataset 1")
lr_d2_bow = evaluate_and_print(lr_model, X2_train_bow, y2_train, X2_test_bow, y2_test,
"Logistic Regression", "BoW", "Dataset 2")
lr_d2_tfidf = evaluate_and_print(lr_model, X2_train_tfidf, y2_train, X2_test_tfidf, y2_test,
"Logistic Regression", "TF-IDF", "Dataset 2")
```

```
from sklearn.svm import LinearSVC

svm_model = LinearSVC()

svm_d1_bow = evaluate_and_print(svm_model, X1_train_bow, y1_train, X1_test_bow,
y1_test, "Linear SVC", "BoW", "Dataset 1")
svm_d1_tfidf = evaluate_and_print(svm_model, X1_train_tfidf, y1_train, X1_test_tfidf,
y1_test, "Linear SVC", "TF-IDF", "Dataset 1")
svm_d2_bow = evaluate_and_print(svm_model, X2_train_bow, y2_train, X2_test_bow,
y2_test, "Linear SVC", "BoW", "Dataset 2")
svm_d2_tfidf = evaluate_and_print(svm_model, X2_train_tfidf, y2_train, X2_test_tfidf,
y2_test, "Linear SVC", "TF-IDF", "Dataset 2")
```